

35 Projects, 35 Powerful Skills

Beginner-detailed edition. Every project now opens with a plain-English explanation of the core idea before asking you to build anything — no jargon left unexplained. Still no step-by-step tutorials: you get the concept, the spec, the tools, and a solution key to check yourself against.

35

projects

4h

per day

5

sections/card

100%

free & local

Each card has 5 parts: a purple **What Is This?** box explaining the concept simply, a pink **Build** spec (what to make), a **Requirements** list (constraints that force real decisions), **Tools** tags (what to install/use), and an orange **Solution Approach** box at the bottom describing what correct behavior looks like, so you can check your own work without being handed the code.

Adjusted for your setup: every project assumes **Ollama running Llama 3.2** locally, 100% free, no cloud API keys anywhere. A few projects needed real changes to fit that: fine-tuning (Day 10) uses the LoRA → merge → GGUF → Ollama Modelfile pipeline since Ollama can't fine-tune directly; the MCP client (Day 23) is one you build yourself with Ollama's tool-calling, since most ready-made MCP clients are built around Claude; vision (Day 30) needs the separate Llama3.2-vision model since the plain 1B/3B text models can't see images at all. The RAG projects (Days 15-17) now use **Docling** for document parsing, which handles PDFs, tables, and layout far better than plain text extraction, and is a genuinely valuable tool to know.

Roadmap → 35-Day Curriculum Map

Each of the 12 stages from the "Roadmap To Learn Agentic AI" mapped to the specific project days that build it, so you can see exactly where every roadmap topic is covered in the 35-project plan below.

1 Introduction to Agentic AI

Days 1–3 · Structured Output Extractor · Streaming Chatbot · Single-Tool Agent

The first working pieces of any agent: predictable output, live responses, and the model-decides-to-act loop.

2 Fundamentals of AI & ML

Days 8, 11, 31 · Model Size Trade-off Benchmark · Embedding Pipeline · Model Serving & Concurrency

Hands-on grounding in parameters, embeddings, and quantization — the ML fundamentals every later project leans on.

3 Programming & AI Frameworks

Days 1–3, 22–23 · Ollama/pydantic basics (Wk1) · Your First MCP Server · Connect a Real Client

Python libraries for AI agents, function-calling APIs, and building on a real interoperability framework (MCP).

4 Large Language Models (LLMs)

Days 8–9, 30 · Model Size Trade-off Benchmark · Swappable Model Config · Vision

Comparing model sizes, abstracting which model you call, and working with a multimodal (vision) LLM.

5 Understanding AI Agents

Days 3–4, 25 · Single-Tool Agent · Multi-Tool Router · Multi-Agent System

Agent architectures from a single tool call up to multiple specialized agents coordinated by a manager.

6 AI Memory & Knowledge Retrieval

Days 5, 11–14 · Persistent Memory · Embedding Pipeline · Semantic Search · Vector Database · Hybrid Search + Reranking

Short vs. long-term memory, vector databases, and the retrieval techniques that power every RAG system later.

7 Decision-Making & Planning

Days 18, 25–26 · Agentic RAG · Multi-Agent System · Human-in-the-Loop

Letting the model decide when to retrieve, when to delegate, and when to pause for approval before acting.

8 Prompt Engineering & Adaptation

Days 1, 9–10 · Structured Output (format param) · Swappable Model Config · Fine-Tune Llama 3.2 With LoRA

From controlling output shape through prompting, to adapting the model itself via lightweight fine-tuning.

9 Reinforcement Learning & Self-Improvement

Days 7, 10, 19 · Eval Harness for Week 1 · Fine-Tune With LoRA · RAG-Specific Evals

Building the feedback loops (evals, reward-style scoring, fine-tuning) that let a system improve over time.

10 Retrieval-Augmented Generation (RAG)

Days 15–21 · Basic RAG · Chunking Strategy Comparison · Multi-Document RAG · Agentic RAG · RAG-Specific Evals · GraphRAG · Grounded RAG With Confidence

The full RAG arc: grounding answers in real documents, then hybrid retrieval, agentic retrieval, and GraphRAG.

11 Deploying AI Agents

Days 31–32, 34 · Model Serving & Concurrency · Resource & Latency Dashboard · Production Deployment

Scaling for concurrent requests, tracking latency/resource cost, and shipping behind auth and rate limits.

12 Real-World AI Agent Applications

Days 29–30, 35 · Voice Interface · Vision · Flagship Integration

Multimodal, real-world-facing capabilities (voice, vision) brought together in the final flagship system.

Note: several days intentionally serve more than one roadmap stage (e.g. Day 10's fine-tuning touches both "Prompt Engineering & Adaptation" and "Reinforcement Learning & Self-Improvement") — the project work is sequential by week, the roadmap stages are conceptual and some naturally overlap.

W1

Core LLM App Patterns

Days 1-7. The building blocks every agent and RAG system is made of: structured output, tool use, memory, latency, and proving your system actually works.



1 Structured Output Extractor

WHAT IS THIS? When you chat with an LLM normally, it replies with paragraphs of text — great for humans, useless for code. "Structured output" means forcing the model to reply in a fixed, predictable shape (like a JSON object with exact fields) so your program can read the answer automatically, the way it reads data from a database. It's the difference between asking a friend to describe a book out loud, versus handing them a form with boxes for Title, Author, and Pages.

BUILD

A CLI tool that sends messy text to your local Llama 3.2 model through Ollama and forces the reply into a strict JSON schema — e.g. a "recipe extractor" that always returns `{name, ingredients[], steps[]}` no matter how disorganized the input recipe is. Ollama has a `format` parameter built exactly for this — use it instead of just asking nicely in the prompt text.

REQUIREMENTS

- Validate the parsed JSON against a schema (e.g. with `pydantic`) and automatically retry if it's invalid
- Handle two input shapes: pasted text and a `.txt` file path
- No hardcoded model name or file paths — use a config file or CLI args

TOOLS

ollama (python) pydantic json

SOLUTION APPROACH

- Calling Ollama's chat with `format=recipe_schema` should return content that parses directly as valid JSON matching your schema
- Even on a genuinely messy input, output stays well-formed JSON — shape matters more than perfect extraction here
- Skip validation once on purpose and you should be able to reproduce a case of malformed JSON — that's the proof retry logic is needed
- Final output should be a clean Python object, never a raw string still wrapped in ````json` fences

2 Streaming Chatbot

WHAT IS THIS? Normally your program waits — sometimes several seconds — until the *entire* reply is ready before showing anything. Streaming instead shows each word (technically each "token") the instant it's generated, like watching someone type live instead of waiting for them to hit send. It doesn't make the model faster overall, but it makes your app feel dramatically faster.

BUILD

A terminal chatbot where the reply prints token-by-token as it's generated, using Ollama's `stream=True` mode and iterating over the generator it returns, instead of waiting for one finished response object.

REQUIREMENTS

- Must use real streaming, not chunking a finished response to fake it
- Let the user press `Ctrl+C` mid-stream and exit that response cleanly, no stack trace
- Show something on screen within 1 second, even before the first real token

TOOLS

ollama (python) `sys.stdout.flush` rich (optional)

SOLUTION APPROACH

- You should be able to visibly see words appear incrementally — if it looks identical to a non-streamed version, it's not wired correctly
- A correct `Ctrl+C` handler catches the interrupt around just the streaming loop, so you can stop one answer and immediately ask another question
- Expect roughly 15–40 tokens/sec on `llama3.2:3b` on typical consumer hardware — write down your own number as a baseline for later days

3 Single-Tool Agent

WHAT IS THIS? LLMs are great at language but bad at exact math, live facts, or taking real actions. "Tool use" fixes this: you describe a function to the model (like "a calculator that takes two numbers"), and the model can ask your code to run it, then use the result in its answer. Model asks → your code runs it → model reads the result — this cycle is the "agent loop," the backbone of every agent you'll ever build.

BUILD

An agent that looks at a question and decides for itself, using Ollama's tool-calling support, whether to answer directly or call a calculator tool. Run the loop: model requests the tool with arguments → your Python code executes it → result goes back to the model → model gives its final answer.

REQUIREMENTS

- The decision to call the tool must come from the model's `tool_calls` field, not a keyword check you wrote

- Handle malformed tool arguments (e.g. non-numeric input) without crashing
- Print both the tool call made and the final answer so you can see the reasoning

TOOLS

ollama (tools=) plain python function

SOLUTION APPROACH

- llama3.2:3b supports Ollama's tool-calling reasonably well; llama3.2:1b is noticeably less reliable at it — that comparison is itself useful data to note
- On "what's 847 times 39, plus 12" the model should emit a tool call, receive 33045 back, then state that number — not attempt the arithmetic in text itself
- On "what's the capital of France," a correct agent answers directly with zero tool calls
- If it always calls the tool regardless of the question, that's a bug in your tool's description, not a model limitation

4 Multi-Tool Router

WHAT IS THIS? Once an agent has more than one tool, it has to figure out which one fits a given question — this is "routing." It works exactly like Day 3, just with the model choosing among several options. Your tool's text description is what the model uses to decide, so a vague description causes wrong choices, the same way a badly labeled button confuses a human.

BUILD

Extend Day 3 into an agent with three tools — a calculator, a unit converter, and a date/time tool — where the model picks the correct one (or none) per question.

REQUIREMENTS

- Each tool needs a distinct, specific description and parameter schema, not just a name
- Test one deliberately ambiguous question that could plausibly use two tools (e.g. mixing date math with unit conversion)
- Log every tool call made, even ones you didn't expect

TOOLS

ollama (tools=) 3 small python functions

SOLUTION APPROACH

- Expect llama3.2:3b to route simple single-tool questions correctly almost every time; the ambiguous question is the real test, and partial success there is expected and fine
- A well-built router can chain tool calls when needed (e.g. date tool first, then feed that into unit converter) rather than needing you to pre-chain them in code
- If it consistently picks the wrong tool for an obviously single-purpose question, fix that tool's description first — don't assume you need a bigger model

5 Agent With Persistent Memory

WHAT IS THIS? By default, an LLM has zero memory between separate runs of your program — restart it, and it's a stranger again. "Memory" means saving important information to a real file or database, then loading it back in and including it in your next request, so the model appears to remember things across sessions, the way an app remembers your login.

BUILD

Give your agent memory that survives closing and reopening the terminal, stored in a local SQLite database instead of a Python list that disappears when the script ends.

REQUIREMENTS

- Memory must survive fully restarting the script
- The agent must recall a specific fact from a previous, separate run
- Provide a command to clear/reset memory

TOOLS

sqlite3 (built-in) ollama

SOLUTION APPROACH

- Correct test: say "my favorite language is Rust," fully quit, reopen fresh, ask "what's my favorite language?" — it should answer correctly by reading SQLite, not because the process never actually restarted
- A simple correct design stores (role, content, timestamp) rows and rebuilds the message history on each run; a more advanced version summarizes old memory instead of replaying it verbatim as it grows
- If memory breaks after restarting your whole machine, check you're not accidentally using SQLite's in-memory mode (:memory:) instead of a real file path

WHAT IS THIS?

"Latency" is how long you wait for a response. Local models must load into your computer's memory before they can respond quickly — the very first request after starting Ollama is always slower ("cold start") than every request after ("warm"). Understanding this timing, and how to keep a model warm and ready, is the first step to an app that feels snappy instead of sluggish.

BUILD

Since everything runs locally and free, "cost" isn't dollars — it's your machine's time. Build an instrumentation layer around one agent that logs tokens generated, tokens/sec, and wall-clock latency for every call, then use Ollama's `keep_alive` setting to see how much staying loaded actually saves.

REQUIREMENTS

- Log timestamp, prompt length, response length, and duration for every call to a file
- Compare a cold start (model not yet loaded) vs. a warm request (already loaded via `keep_alive`)
- Report your own measured tokens/sec, not a number from documentation

TOOLS

ollama python time module pandas (optional)

SOLUTION APPROACH

- Expect a large gap between the very first request after starting Ollama (model loads into memory) and every request after — often several seconds vs. under a second just to start generating
- Correct instrumentation logs numbers automatically on every call, not numbers you eyeballed once
- If warm and cold numbers look identical, check `ollama ps` — the model may already have been loaded before your test started

WHAT IS THIS?

"Evals" (evaluations) are automated tests built specifically for AI systems — instead of checking that code does the exact same thing every time, you're checking whether an inherently unpredictable system is **usually** giving good answers. Without evals you're stuck manually reading outputs and guessing whether a change helped — evals turn that guess into a number you can track.

BUILD

An evaluation script that runs your Day 3/4 agent against 10-15 fixed test questions and scores the outputs automatically every time, so you can tell if a change improved or broke something without re-reading transcripts by hand.

REQUIREMENTS

- Include code-based checks (right tool called? correct number in the response?) and a separate LLM-as-judge check using Llama 3.2 itself as the judge
- Store test cases as data (a JSON/YAML file), not hardcoded inline
- Output a clear pass/fail report with a total score

TOOLS

ollama (agent + judge) pytest (optional) json / yaml

SOLUTION APPROACH

- A correct check for `"847*39+12"` extracts the number from the response and compares it numerically — it doesn't just search for the exact string `"33045,"` since formatting varies
- Using the same small local model as judge is a known weak point — expect occasional inconsistency between runs on subjective questions; that inconsistency is itself a lesson about trusting small local judges
- Re-running an unchanged agent twice should give the same code-based scores both times, though the LLM-judge score may shift slightly — record both

W2

Model Trade-offs & Retrieval Infra

- Days 8-14. Compare model sizes, fine-tune Llama 3.2 yourself, and build the embeddings/vector-DB/search infrastructure that RAG depends on.



8 Model Size Trade-off Benchmark

WHAT IS THIS? AI models come in different sizes, measured in "parameters" (Llama 3.2 has a 1-billion and a 3-billion parameter version). Bigger generally means smarter but slower and heavier; smaller means faster and lighter but more mistake-prone on complex tasks. There's no universal "best" — the right size depends on what you're building, and this project is about generating your own evidence instead of trusting a claim.

BUILD

Pull both llama3.2:1b and llama3.2:3b via Ollama and run your Day 7 eval harness against each. This is about seeing the classic small-model trade-off with your own numbers, not someone else's benchmark chart.

REQUIREMENTS

- Run the exact same eval set from Day 7 against both model sizes
- Record tokens/sec, RAM usage (via `ollama ps`), and eval pass rate for each
- Write a short conclusion: which model for which kind of task, and why

TOOLS

ollama pull your Day 7 harness ollama ps

SOLUTION APPROACH

- Expect 1b to be noticeably faster and lighter but fail more of your structured-output/tool-calling tests than 3b — the gap usually shows up most in the tool-calling agent, less in simple Q&A
- If both score identically, your test questions are too easy to reveal a difference — add 2-3 harder cases (multi-step reasoning, ambiguous tool choice) and rerun
- A correct write-up names a specific real use case for each size, not just "bigger is better"

9 Swappable Model Config

WHAT IS THIS? As your code grows, if the specific model name ("llama3.2:3b") is scattered across many files, switching models later becomes a painful find-and-replace job. "Abstraction" hides that detail behind one generic interface, so the rest of your program never needs to know or care exactly which model is underneath.

BUILD

Refactor one earlier agent so which local model it uses (llama3.2:1b, llama3.2:3b, or another) is controlled by a single config value — no code changes needed to switch.

REQUIREMENTS

- One function/class is the only place that knows the model name; everything else calls a generic interface
- Switching via config/env var, then rerunning, works with zero other file changes
- Document one task where the two models genuinely disagree in output

TOOLS

ollama python-dotenv or config.yaml

SOLUTION APPROACH

- Correct abstraction looks like one `get_model_client(config)` function or a thin wrapper class — if switching still means editing more than one line in one file, it's not done yet
- Test it by actually flipping the config twice and confirming both runs produce visibly different-style output

10 Fine-Tune Llama 3.2 With LoRA

WHAT IS THIS? Prompting changes *what you ask* a model; fine-tuning changes the model itself by adjusting its internal weights. Full fine-tuning of every weight is extremely expensive — LoRA ("Low-Rank Adaptation") trains a small set of extra weights instead of touching the whole model, making it realistic even on a personal computer. Think of it as adding a small removable attachment rather than rebuilding the whole machine.

BUILD

Fine-tune Llama 3.2 (1B or 3B) on a tiny dataset you build yourself — 50-100 examples in a specific style or Q&A format. Ollama doesn't fine-tune directly, so the real pipeline is: train a LoRA adapter outside Ollama → merge it into the base weights → convert to GGUF format → import into Ollama with a custom Modelfile so it runs like any other model.

REQUIREMENTS

- Must use LoRA/PEFT, not full fine-tuning — research why that matters at this size on consumer hardware
- End state must be runnable via `ollama run your-custom-model`, not just a training checkpoint
- Compare fine-tuned vs. base model output on 5 held-out examples it never trained on

TOOLS

unsloth or peft+transformers

llama.cpp convert script

Ollama Modelfile

SOLUTION APPROACH

- Unsloth specifically supports Llama 3.2 and is much faster/lighter than raw PEFT+transformers at this size — check it before building a training loop from scratch
- `ollama list` should show your new custom model name after import, and `ollama run your-custom-model` should respond in your trained style more consistently than the base model
- If output looks identical to the base model, common causes are LoRA rank too low, too few training steps, or the adapter never actually got merged before GGUF conversion

11 Embedding Pipeline

WHAT IS THIS? An embedding turns text into a list of numbers (a "vector") that captures its *meaning*, positioned so similar-meaning texts land numerically close together — even with zero shared words. This is the trick that lets a computer know "puppy" and "dog" are related without you ever hardcoding that.

BUILD

A script that turns any block of text into a vector embedding using a dedicated local embedding model pulled via Ollama (e.g. `nommic-embed-text` — `llama3.2` itself isn't built for this).

REQUIREMENTS

- Must work on both a short sentence and a full page of text
- Print cosine similarity scores for 3 sentence pairs you choose, and sanity-check the numbers make sense

TOOLS

ollama embeddings

numpy

nommic-embed-text

SOLUTION APPROACH

- Two paraphrases ("I love hiking on weekends" / "I enjoy hikes when I'm off work") should score noticeably higher than two unrelated sentences — if not, check the similarity math, not the model
- Cosine similarity is $\text{dot}(a, b) / (\text{norm}(a) * \text{norm}(b))$ — values wildly outside roughly -1 to 1 mean a math bug
- `nommic-embed-text` returns 768-dimension vectors — printing the vector's length is a quick sanity check you're getting real embeddings back

12 Semantic Search Over Your Own Notes

WHAT IS THIS? Keyword search (Ctrl+F) only finds exact word matches. Semantic search compares the *meaning* of your query against the meaning of your documents using embeddings, so it can find genuinely relevant results even when the wording is completely different from what you typed.

BUILD

A search tool over a folder of your own real text files that finds relevant results by meaning rather than exact match, using the Day 11 embedding pipeline.

REQUIREMENTS

- Must beat plain keyword search on 3 test queries where the exact words don't appear in the target file but the meaning does
- Return results ranked by similarity score, not file order

TOOLS

ollama embeddings

numpy

Day 11 code

SOLUTION APPROACH

- Example working test: a file about "budgeting for a trip to Japan" that never uses the word "vacation" should still surface when you search "vacation planning"
- If keyword and semantic search return identical rankings on every test query, your queries aren't actually testing meaning — use paraphrases, not reused vocabulary

13 Add a Real Vector Database

WHAT IS THIS? Once you have thousands of embeddings, searching through all of them one by one gets slow. A vector database is specialized storage built to find the closest-matching vectors very quickly, even across millions of entries — the same way a regular database makes normal lookups fast.

BUILD

Swap your in-memory embedding list for ChromaDB, a local vector database that runs embedded with zero setup, so your search tool scales past a few hundred documents without re-embedding everything every run.

REQUIREMENTS

- Must support adding/removing a single document without rebuilding the entire index
- Benchmark query speed at ~100 documents vs. ~1,000+ documents and report both

TOOLS

chromadb ollama embeddings

SOLUTION APPROACH

- Chroma's `collection.add().delete()` should let you add or remove one document and query immediately, no full rebuild step
- Query latency should grow much slower than linearly as document count rises — that's the point of a vector index; if 1,000 docs is ~10x slower than 100, you may be brute-force scanning by mistake

14 Hybrid Search + Reranking

WHAT IS THIS? Semantic search is great at meaning but sometimes misses exact terms (a product code, a name) keyword search would catch instantly. Hybrid search runs both and combines results. Reranking is a separate, more careful second pass that re-scores a shortlist to put the truly best results on top — a fast resume filter, then a careful interview for the finalists.

BUILD

Combine keyword search (BM25) and semantic search (Day 13) into one hybrid system, then add a reranking step using a small local cross-encoder model that reorders the combined candidates by actual relevance.

REQUIREMENTS

- Show a real case where semantic-only search fails (misses an obvious keyword match) and hybrid fixes it
- Reranking must be a distinct second pass, not merged into initial retrieval

TOOLS

rank_bm25 chromadb sentence-transformers cross-encoder

SOLUTION APPROACH

- Correct pipeline: top-20 from BM25 + top-20 from semantic search → merge/dedupe → rerank the ~30-40 combined candidates → return top 5
- Hybrid typically beats semantic-only on an exact term (product code, acronym, proper noun) the embedding model doesn't weight heavily

W3

RAG, Properly (with Docling)

Days 15-21. From basic grounded Q&A to agentic retrieval, GraphRAG, and evals that measure retrieval and generation as two separate failure points.



15 Basic RAG

WHAT IS THIS? RAG = Retrieval-Augmented Generation. Instead of relying only on what a model memorized in training (which may be outdated, wrong, or simply not about your documents), RAG retrieves relevant chunks of your own real documents first, then hands them to the model as extra context before asking it to answer — like giving someone the exact manual page to read before asking them a question, instead of expecting them to recall it from memory.

BUILD

A question-answering system grounded in one real document you actually care about. Use Docling to parse it properly (it handles PDF layout, tables, and structure far better than a naive text dump), chunk it, embed the chunks with Ollama, and answer questions using only retrieved content.

REQUIREMENTS

- Every answer must be traceable to a specific chunk of the source document
- Must correctly say "I don't know" when the document doesn't contain the answer

TOOLS

[docling](#) [ollama](#) [chromadb](#)

SOLUTION APPROACH

- Docling's DocumentConverter should turn a PDF into structured content (headings, tables, reading order preserved) far more cleanly than raw text extraction — garbled, out-of-order chunks mean you're not using its structured output correctly
- A correct grounded answer names its source (page number, heading, or raw chunk text) — an answer with no traceable source isn't actually grounded even if it's correct
- Test one question the document truly can't answer — a correct system admits it doesn't know instead of confidently hallucinating

16 Chunking Strategy Comparison

WHAT IS THIS? You can't hand an entire book to a model at once. "Chunking" splits documents into smaller pieces before embedding and storing them, so retrieval can grab just the relevant piece. *How* you split matters a lot — cutting a table or sentence in half mid-way can destroy its meaning.

BUILD

Run the same 10 questions through Day 15's RAG using three chunking approaches: Docling's built-in structure-aware HybridChunker, a naive fixed-size character splitter, and a sentence-based splitter. Compare which actually answers more correctly.

REQUIREMENTS

- Log which strategy answers more of your 10 questions correctly
- Write a short note on why the winner won, based on your document's actual structure

TOOLS

[docling HybridChunker](#) [langchain text splitters](#)

SOLUTION APPROACH

- Docling's structure-aware chunker should generally win on documents with real structure (headings, tables) since it avoids cutting mid-table or mid-sentence
- If all three score identically, your source document is probably too simple to show a difference — try a technical manual or something with tables

17 Multi-Document RAG

WHAT IS THIS? Metadata is "data about your data" — which file a chunk came from, what date, what category. Storing this alongside each chunk lets you narrow a search before it even runs ("only 2024 reports") instead of searching everything and hoping the right answer surfaces.

BUILD

Extend RAG to search across 10+ documents of mixed formats (PDF, Word, plain text) using Docling to parse each format consistently, with metadata (filename, section) attached to every chunk.

REQUIREMENTS

- Must support filtering retrieval by metadata (e.g. only a specific file or section)
- Test one question that genuinely requires pulling info from two different documents

TOOLS

[docling \(PDF/DOCX/PPTX\)](#) [chromadb metadata filters](#) [ollama](#)

SOLUTION APPROACH

- Docling's converter should handle PDF and DOCX through one unified interface — separate parsing code per file type means you're not using its real value
- Chroma's where filter should restrict a query to a document subset without touching your embedding logic at all
- The multi-document test question should fail or answer incompletely if you retrieve from only one document — that's the proof it's genuinely combining sources

18 Agentic RAG

WHAT IS THIS? Basic RAG always retrieves exactly once, needed or not. Agentic RAG treats retrieval itself as a tool (Day 3's pattern) the model can choose to call — zero times for small talk, multiple times if one search reveals it needs to look up something else too.

BUILD

Give your RAG system agency: using Ollama tool calling, let the model decide whether it needs to retrieve, how many times, and when it has enough to answer.

REQUIREMENTS

- Handle one question needing two separate retrieval calls to fully answer
- Must NOT retrieve at all for something that doesn't need it (e.g. "hi, how are you")

TOOLS

ollama (tools=) **Day 15-17 RAG as a tool**

SOLUTION APPROACH

- A correct build logs zero tool calls for small talk and one-or-more for real questions — always retrieving regardless means your code is forcing it, not the model deciding
- On the two-step question, logs should show retrieve → read result → retrieve again with a refined query — that sequential dependency is the actual "agentic" part

19 RAG-Specific Evals

WHAT IS THIS? A RAG system can fail in two separate places: it might retrieve the wrong information (retrieval failure), or retrieve the right information but the model still gets the answer wrong (generation failure). Testing both separately tells you which half to fix.

BUILD

A proper eval suite for your RAG system specifically, measuring retrieval quality and generation quality separately.

REQUIREMENTS

- Retrieval eval: for each test question, was the correct chunk actually in the retrieved set?
- Generation eval: given the correct chunk, is the answer faithful to it — a separate check

TOOLS

ollama (LLM-as-judge) **hand-labeled test set**

SOLUTION APPROACH

- You need a small set of questions with manually written ground-truth chunks and answers — without that, you can't measure precision/recall, only vibes
- A correct faithfulness check re-reads the chunk and the answer together and asks "is every claim supported here?" — not the weaker "does this sound right?"
- Expect to find cases where retrieval succeeded but generation still failed — that distinction is exactly what this eval is built to catch

20 GraphRAG

WHAT IS THIS? Some questions need connecting multiple separate facts ("who manages the person who wrote this report") — plain similarity search struggles because no single chunk contains the full answer. A knowledge graph stores facts as connected nodes, letting you follow a chain of relationships to answer these "multi-hop" questions.

BUILD

Extract entities and relationships from your documents into a simple knowledge graph, and use it to answer a multi-hop question that plain chunk-retrieval genuinely struggles with.

REQUIREMENTS

- Correctly answer at least one multi-hop question your Day 15-17 vector-only RAG gets wrong
- Graph must come from your own real documents' extracted entities, not hand-typed

TOOLS

ollama (structured extraction)

networkx

SOLUTION APPROACH

- Entity extraction should reuse Day 1's structured-output pattern: ask for {entities:[...], relationships:[...]} in a fixed schema
- A correct multi-hop answer traces an actual path through your graph — if you can't point to the specific edges that produced the answer, the graph isn't really being used

21 Grounded RAG With Confidence

WHAT IS THIS? "Grounding" means an answer is actually backed by real retrieved evidence, not just generated from the model's general training. A visible confidence signal helps a user know when to trust an answer versus double-check it — a doctor saying "I'm fairly sure" versus "I'd want more tests."

BUILD

Add citations and a confidence signal to your RAG answers, so every response shows exactly which chunk it came from and how sure the system is.

REQUIREMENTS

- Every answer must include a real, checkable source citation, not prose claiming to be grounded
- Low-confidence answers must be visibly flagged differently, not just logged internally

TOOLS

ollama

chroma similarity scores

SOLUTION APPROACH

- The top retrieved chunk's similarity score is a simple, defensible confidence proxy — very low similarity should visibly downgrade the answer, not just sit in a log
- Test with one well-covered question and one clearly uncovered one — the output should look noticeably different, not identical with a hidden number attached

W4

MCP & Multi-Agent Orchestration

Days 22-28. Build real MCP servers and your own Ollama-based client, coordinate multiple agents, and add the safety/observability layers production systems need.



22 Your First MCP Server

WHAT IS THIS? MCP (Model Context Protocol) is a standardized way for AI apps to discover and use external tools/data, so any tool built to the MCP spec works with any MCP-compatible app instead of every company inventing its own incompatible plugin system. An "MCP server" is your side — it advertises what it offers and knows how to run it when asked.

BUILD

Build an MCP server exposing one of your existing tools — the Day 3 calculator or Day 15 RAG system — using the official MCP Python SDK, so any real MCP client can discover and call it.

REQUIREMENTS

- Must follow the actual MCP spec (discovery, schema, invocation) via the official SDK, not a REST wrapper you're calling "MCP"
- Test by connecting a real client to it and calling the tool through that client

TOOLS

mcp (official SDK) fastmcp (simpler wrapper)

SOLUTION APPROACH

- fastmcp is the easiest path if the raw SDK feels heavy — a decorator turns a plain Python function into a spec-compliant MCP tool
- Since you're on Ollama, most polished MCP clients (Claude Desktop) are built around Claude, not local models — for testing, plan to build your own minimal client in Day 23
- Running your server, any spec-compliant "list tools" call should return your tool's name, description, and schema exactly as you defined them

23 Connect a Real Client

WHAT IS THIS? The "client" side of MCP is whatever asks a server what tools it has, relays those to an AI model, and carries the model's tool requests back to the server. Building your own client teaches you the full request/response cycle instead of treating it as a black box.

BUILD

Build your own minimal MCP client that uses Ollama's tool-calling to discover tools from your Day 22 server and call them — the honest local-only version of "connect a client," and arguably a better learning exercise than configuring someone else's app.

REQUIREMENTS

- Must call the server's tool-listing endpoint to discover tools dynamically, not hardcode the schema
- Feed discovered schemas into Ollama's tools=, and correctly route Ollama's tool_call responses back through MCP
- Test with 3 different requests through the full round trip

TOOLS

mcp (client SDK) ollama (tools=)

SOLUTION APPROACH

- Correct flow: client asks server "what tools?" → server returns schemas → client passes them to Ollama's tools= → Ollama decides to call one → client re-sends that call to the server → server executes and returns a result → client feeds it back to Ollama for a final answer
- If you're hardcoding a tool's parameters instead of reading them from the server's response, dynamic discovery isn't really implemented
- Success: changing your Day 22 server's tool (renaming a parameter, adding a tool) requires zero client code changes

24 MCP-Wrapped RAG

WHAT IS THIS? This is simply Day 15's RAG system exposed through Day 22's MCP server — your RAG code stops being something only your own script can call directly, and becomes a tool any MCP-compatible application could plug into, the way a USB port lets any compatible device connect regardless of who made it.

BUILD

Wrap your Day 15-17 RAG system as an MCP tool on your server, so your Day 23 client (or any MCP client) can ask document-grounded questions through the protocol instead of calling your RAG code directly.

REQUIREMENTS

- The RAG tool's schema should accept a question and return an answer with its source citation
- A generic client should query it with zero RAG-specific code — it only needs to know "there's a tool that answers questions"

TOOLS

mcp docling chromadb ollama

SOLUTION APPROACH

- No new mechanics beyond Days 22-23 — success is architectural: RAG logic lives entirely inside the MCP server, invisible to whatever calls it
- You should be able to swap your RAG internals (e.g. try a different Day 16 chunking strategy) without touching the MCP interface or client at all

25 Multi-Agent System

WHAT IS THIS? Instead of asking one AI to do an entire complex task, you can split it into specialized roles — one agent good at finding information, another good at writing — coordinated by a manager that decides which to call and when, mirroring how a human team divides labor.

BUILD

Build a system with 2+ specialized agents coordinated by a manager agent — e.g. a researcher agent (uses your RAG/MCP tools) and a writer agent — with the manager deciding which to invoke.

REQUIREMENTS

- Manager must delegate deliberately based on the request, not call both agents unconditionally every time
- Handle one agent failing without crashing the whole system

TOOLS

ollama (different system prompts per role) your RAG/tools

SOLUTION APPROACH

- With one small local model, each "agent" is realistically the same llama3.2 with a different system prompt/toolset, orchestrated by your own Python — that's a legitimate, common pattern
- A research-only request should never invoke the writer; a request needing both should show researcher → writer in that order
- Forcing the researcher to fail should produce a graceful "couldn't find relevant information" path, not a crash or a hallucinated answer

26 Human-in-the-Loop

WHAT IS THIS? Some actions an agent might take are risky or hard to undo. "Human-in-the-loop" means the agent pauses before doing anything like that, explains what it's about to do, and waits for a real person to approve or reject it — a safety checkpoint, like requiring manager approval before a large bank transaction.

BUILD

Add an approval step to one agent so before a risky/irreversible action (deleting a memory entry, "sending" something), it pauses, clearly states what it's about to do, and waits for approval.

REQUIREMENTS

- Must describe the pending action in plain language before pausing
- Support both "approve" and "reject with feedback," acting correctly on each

TOOLS

ollama your agent's action layer

SOLUTION APPROACH

- The pause must sit between "model decided to call the risky tool" and "tool actually executes" — if the tool already ran by the time you're asked, the safety mechanism doesn't work
- Rejecting with feedback should visibly change the agent's next attempt (e.g. "no, delete a different entry" correctly targets the right one), not just give up

27 Guardrails

WHAT IS THIS? Guardrails are checks that run *around* the model — before its input and after its output — catching problems the model itself might miss if you only ask nicely in the prompt. A dedicated inspector, rather than trusting the same person doing the work to also grade their own homework.

BUILD

Add input and output guardrails to an agent — block obvious PII in inputs, block unsafe/off-topic content in outputs. Ollama can serve llama-guard (Meta's dedicated moderation model) alongside llama3.2, giving a real second-model safety check.

REQUIREMENTS

- Must catch a deliberately bad test input you design yourself (a fake but realistic phone number/email)
- The check must be a separate step/model call, not extra wording in the main system prompt

TOOLS

ollama (llama-guard3)

regex (first-pass PII)

SOLUTION APPROACH

- Layered is correct: cheap regex catches obvious PII instantly, llama-guard catches subtler/contextual cases regex can't
- Test one input regex should catch and one subtler input only the moderation model should catch — a correct system flags both, through different mechanisms
- An in-prompt-only "guardrail" typically fails under adversarial phrasing — demonstrating that failure first is worth doing

28 Observability

WHAT IS THIS? Observability means seeing, after the fact, exactly what your AI system did and why — every model call, every tool call, how long each took — recorded somewhere searchable, instead of relying on your memory of watching it run once in a terminal.

BUILD

Add tracing to one bigger project so every model/tool call, plus latency, token counts, and errors, is recorded in a way you can query after the fact.

REQUIREMENTS

- Must be able to answer "why did it do that?" for a past run from logs alone, no rerunning
- Logs must be structured and queryable, not raw console output

TOOLS

sqlite3 / JSON lines

Langfuse (self-hosted, free)

SOLUTION APPROACH

- Self-hosted Langfuse (via Docker) gives a real trace-viewer UI, exactly what production teams use — worth trying once even at small scale, though a hand-rolled SQLite logger is a completely legitimate simpler alternative
- Correct test: reconstruct a full past run's sequence using only your logs, not memory — if you can't, the logs aren't detailed enough

W5

Advanced & Shipping

Days 29-35. Multimodal input, real model serving, resource engineering, and one flagship system that ties everything together.



29 Voice Interface

WHAT IS THIS? Adding voice means chaining three separate pieces: speech-to-text (spoken words → text), your existing text-based agent (unchanged), and text-to-speech (agent's text reply → spoken audio). None of the three need to know about each other beyond passing text back and forth.

BUILD

Add a fully local, free voice layer to one agent: speech in, agent processes as normal, speech out. No cloud voice APIs.

REQUIREMENTS

- Must handle a follow-up question in the same voice session, not just one-shot
- Should work fully offline (mic in, speaker out)

TOOLS

faster-whisper Piper TTS ollama

SOLUTION APPROACH

- faster-whisper's smaller sizes (tiny/base) transcribe fast enough to feel real-time on modest hardware; larger sizes are more accurate but slower — the same trade-off as Day 8, now applied to a different model type
- Correct multi-turn test: ask by voice, get a spoken answer, then ask "what about for a different number" without restating context — your Day 5 memory pattern should carry it

30 Vision

WHAT IS THIS? Vision-capable ("multimodal") models can accept an image as input, not just text, and reason about what's actually in the picture — a fundamentally different architecture from a text-only model, which is why your regular llama3.2 can't do this at all.

BUILD

Give an agent the ability to look at an image and answer questions about it, using Ollama's llama3.2-vision model (not plain llama3.2, which is text-only).

REQUIREMENTS

- Test with an image where the answer genuinely requires visual understanding (chart values, spatial layout), not something OCR alone could answer
- Handle an image with no relevant question, or a question the image can't answer

TOOLS

ollama pull llama3.2-vision:11b Pillow

SOLUTION APPROACH

- llama3.2-vision is a much larger download and noticeably slower than plain llama3.2 — a real hardware/patience trade-off worth noting as data
- A correct test uses a question the model must actually "look" to answer, e.g. "how many people are in this photo" — a question answerable from filename/metadata alone doesn't prove vision is working
- If responses ignore the image, check you're sending it as part of the message content in the format Ollama's vision models expect

31 Model Serving & Concurrency

WHAT IS THIS? "Quantization" shrinks a model's size and speeds it up by storing its internal numbers with less precision, trading a bit of accuracy for a lot of speed/memory savings. Ollama already quantizes by default — this project is about understanding that trade-off and how Ollama handles multiple requests hitting the same model at once.

BUILD

Configure Ollama to handle multiple concurrent requests properly and measure what actually happens to latency and quality under load, plus compare a couple of quantization levels of llama3.2.

REQUIREMENTS

- Send at least 2 truly concurrent requests (not sequential) and observe how Ollama handles them
- Compare behavior with different OLLAMA_NUM_PARALLEL settings
- Compare quantization tags (e.g. q4 vs q8) on speed and, informally, output quality

TOOLS

ollama server env vars python asyncio

SOLUTION APPROACH

- By default Ollama queues requests to one instance; raising `OLLAMA_NUM_PARALLEL` should produce a measurable difference in total time for 2 concurrent requests
- q4 should be noticeably faster and lighter than q8 at some quality cost — the difference is often subtle on simple questions and more visible on precision-demanding ones; try one of your Day 7 eval questions on both

32 Resource & Latency Dashboard

WHAT IS THIS? When you're not paying per API call, "cost" shows up as CPU, memory, and time instead of dollars. Tracking these numbers the same way a cloud service tracks billing is what lets you make informed trade-off decisions instead of guessing.

BUILD

Build a small dashboard comparing your models (1b, 3b, others you've pulled) on latency, memory footprint, and Day 7 eval pass rate, all in one place.

REQUIREMENTS

- Use real numbers pulled from your own logs/benchmarks across Days 6, 8, 28
- Include a case where the "best" model on quality isn't the fastest or lightest, and make that trade-off visible

TOOLS

pandas **streamlit or matplotlib** **psutil (optional)**

SOLUTION APPROACH

- This should mostly assemble data you already generated on Days 6, 8, and 28 — if starting from zero here, go back and confirm those logs were actually saved to files
- A correct dashboard makes the trade-off visually obvious (e.g. a scatter plot of speed vs. eval score) rather than a table you have to mentally compare

33 Semantic Caching

WHAT IS THIS? If two questions are nearly identical in meaning, generating a fresh answer both times wastes time. Semantic caching checks whether a new question is meaningfully similar (via embeddings) to one you've already answered, and reuses that answer if so — the trick is setting the similarity bar correctly.

BUILD

Add a caching layer that recognizes semantically similar queries and reuses cached answers instead of calling the model again — even free local inference still costs real time.

REQUIREMENTS

- Must distinguish a truly similar query from a superficially similar but different one (e.g. "cancel my order" vs. "cancel my subscription" should NOT share a cache entry)
- Report latency savings on a realistic batch of repeated-ish queries you construct

TOOLS

GPTCache **Day 11 embeddings + Day 13 vector DB**

SOLUTION APPROACH

- Try GPTCache first — it's built specifically for this and handles similarity-threshold tuning; comparing your own hand-rolled version afterward checks your understanding
- The similarity threshold is the whole game: too loose causes wrong-answer cache hits, too tight causes rare hits — demonstrate you found a threshold avoiding both failure modes on your own test queries

34 Production Deployment

WHAT IS THIS? Deploying "to production" means making your project reachable as a real running service rather than something only you can run — and the moment it's reachable by other devices, it needs authentication (proving who's allowed) and rate limiting (stopping one user from overwhelming it).

BUILD

Deploy your best project behind a real local API with authentication and rate limiting — even on your own machine, exposing it unprotected the moment it's reachable from another device is a real security gap.

REQUIREMENTS

- Must reject requests without a valid API key
- Must rate-limit a single client

TOOLS

SOLUTION APPROACH

- A correct auth check returns 401/403 for a request with a missing/wrong key — test with an actual curl command missing the header
- A correct rate-limit test fires more requests than allowed in a short window and confirms excess ones are rejected (typically 429), not silently queued

35 **Flagship Integration**

WHAT IS THIS? This final project has no new concept of its own — it's the practice of taking several separately-working pieces you built on different days and making them function together as one coherent system, which in real engineering work is often harder than building any single piece alone.

BUILD

Combine your strongest pieces — likely your MCP server (Days 22-24), agentic RAG (Day 18), and guardrails (Day 27) — into one integrated system on llama3.2 via Ollama, and write a short technical case study on the architecture, the trade-offs of running a small local model, and what you'd change next time.

REQUIREMENTS

- Must actually integrate 3+ previous projects working together, not just describe them side by side
- Case study must include something that didn't work on the first try and how you fixed it

TOOLS

everything from Days 1-34

SOLUTION APPROACH

- There's no single correct architecture — success means someone else could read your case study and understand exactly which projects compose the final system and how data flows between them
- An honest case study names at least one place llama3.2's small size was a real limitation (tool-calling reliability, reasoning depth, context window) — if everything reads as flawless, you probably haven't stress-tested it enough yet